

Elementos funcionales en lenguajes no funcionales

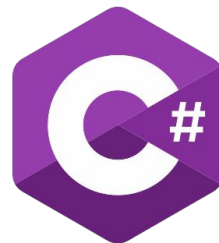
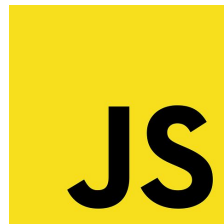
Introducción

Algunos conceptos y características del paradigma funcional se han ido agregando con el tiempo a lenguajes de los paradigmas convencionales.

Permite mejorar la legibilidad del código, su entendimiento, y facilitar su verificación, entre otros beneficios.

Algunos ejemplos incluyen:

- Funciones de orden superior
- Funciones como ciudadano de primera clase
- Expresiones lambda
- Evaluación de subprogramas de forma parcial
- Evaluación perezosa



Java y C#

Funciones de orden superior

Java no brinda soporte para que un método pueda recibir otro método como parámetro o retornar un método como resultado. Las funciones no existen de forma independiente, sino que deben pertenecer a una clase o a un objeto.

La manera que brinda el lenguaje de aproximación esta característica es a través de **interfaces funcionales** y **clases anónimas**. Las interfaces funcionales tienen un único método abstracto.

```
@FunctionalInterface
interface Function<T, R> { R apply(T t); }

@FunctionalInterface
interface Predicate<T> { boolean test(T t);
}

@FunctionalInterface
interface Consumer<T> { void accept(T t); }

@FunctionalInterface
interface Supplier<T> { T get(); }
```

Un método puede recibir por parámetro un objeto instancia de una clase que implementa una interfaz funcional.

Funciones de orden superior

Un método puede recibir por parámetro un objeto instancia de una clase que implementa una interfaz funcional.

```
import java.util.function.Function;

public class Example {
    static void applyFunction(Function<Integer, Integer> func) {
        System.out.println(func.apply(10));
    }

    public static void main(String[] args) {

        applyFunction(new Function<Integer, Integer>() {
            @Override
            public Integer apply(Integer x) {
                return x * 2;
            }
        }); // Imprime 20
    }
}
```

Se logra pasaje de comportamiento pero no de manera sencilla.



Funciones de orden superior

En C#, el concepto de funciones de orden superior se materializa de forma más directa que en Java gracias a los **delegados** (*delegates*). Un método puede recibir un delegado como parámetro o devolver un delegado como resultado.

Los métodos asignados a un delegate tienen que respetar los tipos de parámetros y retorno

```
using System;

public class Example
{
    static void ApplyFunction(Func<int, int> func)
    {
        Console.WriteLine(func(10));
    }

    public static void Main()
    {
        ApplyFunction(delegate(int x)
        {
            return x * 2;
        }); // Imprime 20
    }
}
```



Funciones como ciudadanos de primera clase

En Java, los métodos no son ciudadanos de primera clase. Sin embargo, el lenguaje cuenta con algunas características que permiten simular esa facilidad.

- Interfaces funcionales.
- Clases anónimas.
- Lambdas.

```
import java.util.Comparator;

public class Example {
    public static void main(String[] args) {
        Comparator<String> comp = new Comparator<String>() {
            public int compare(String a, String b) {
                return a.length() - b.length();
            }
        };

        System.out.println(comp.compare("hi", "hello")); // -3
    }
}
```



Funciones como ciudadanos de primera clase

En C#, los delegados permiten tratar a los métodos como ciudadanos de primera clase, sin necesidad de envolver el comportamiento mediante objetos.

```
using System;

public class PrimeraClaseEjemplo
{
    public static void Main()
    {
        Func<string, string, int> comp = delegate(string a, string b)
        {
            return a.Length - b.Length;
        };

        Console.WriteLine( comp("hi", "hello")); // Imprime -3
    }
}
```



Funciones como ciudadanos de primera clase

C# permite asignar un método (estático o de instancia) convencional directamente a una variable a partir de la facilidad llamada Conversión de Grupo de Métodos (*Method Group Conversion*).

El compilador hace trabajo adicional para brindar mayor facilidad de escritura y flexibilidad

```
using System;
using System.Collections.Generic;

public class Ejemplo
{
    static int Duplicar(int x){ return x * 2;}
    static int Cuadruplicar(int x){ return x * 4;}
    static int Mitad(int x){ return x / 2; }

    public static void Main()
    {
        List<Func<int, int>> operaciones = new List<Func<int, int>>();

        operaciones.Add(Duplicar);
        operaciones.Add(Cuadruplicar);
        operaciones.Add(Mitad);

        foreach (Func<int, int> operacion in operaciones)
        {
            int resultado = operacion(10);
            Console.WriteLine("Resultado: " + resultado);
        }
    }
}
```



Lambdas

En Java, las lambdas son funciones anónimas que se usan para instanciar **interfaces funcionales**. Las interfaces funcionales tienen un solo método abstracto.

Las lambdas permiten escribir código más conciso y legible, especialmente al trabajar con streams y colecciones.

```
Function<String, Integer> length = s -> {  
    int l = s.length();  
    return l;  
};  
  
System.out.println(length.apply("hello")); //
```

5

```
Function<Integer, Integer> square = x -> x * x;  
  
System.out.println(square.apply(4)); // 16
```



Lambdas

```
import java.util.function.Function;
```

```
public class Example {
```

```
    static void applyFunction(Function<Integer, Integer> func) {  
        System.out.println(func.apply(10));  
    }
```

```
    public static void main(String[] args) {
```

```
        applyFunction(new Function<Integer, Integer>() {
```

```
            @Override
```

```
            public Integer apply
```

```
                return x * 2;
```

```
        };
```

```
    }); // Imprime 20
```

```
}
```

```
}
```

```
import java.util.function.Function;
```

```
public class Example {
```

```
    static void applyFunction(Function<Integer, Integer> func)
```

```
    {
```

```
        System.out.println(func. apply(10));
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        applyFunction(x -> x * 2); //20
```

```
    }
```

```
}
```



Lambdas

En Java:

- Las lambdas se especifican con el símbolo flecha (->).
- Del lado derecho de la flecha ->, se puede definir una expresión o un bloque.
- No requieren declarar los tipos de manera explícita, aunque es posible.
- Los chequeos de tipos se realizan verificando que los parámetros y el retorno de la lambda sea compatible con los tipos involucrados en el método abstracto de la interfaz funcional que se implementa.

Lambdas

En C#, las lambdas son funciones anónimas que se usan para instanciar delegados.

```
using System;

public class Example
{
    static void ApplyFunction(Func<int> func)
    {
        Console.WriteLine(func(10));
    }

    public static void Main()
    {
        ApplyFunction(delegate(int x)
        {
            return x * 2;
        }); // Imprime 20
    }
}
```

```
using System;

public class Example
{
    static void ApplyFunction(Func<int, int> func)
    {
        Console.WriteLine(func(10));
    }

    public static void Main()
    {
        // ApplyFunction((int x) => { return x * 2; });
        ApplyFunction(x => x * 2);
    }
}
```



Evaluación parcial de funciones

Java no brinda soporte nativo para lograr la aplicación parcial de métodos. Nuevamente es posible simular esta característica apelando a interfaces funcionales.

```
import java.util.function.BiFunction;
import java.util.function.Function;

public class Example {
    public static void main(String[] args) {

        BiFunction<Integer, Integer, Integer> power = (base, exp) -> (int)Math.pow(base, exp);

        // Aplicación parcial usando exponent = 2
        Function<Integer, Integer> square = base -> power.apply(base, 2);

        System.out.println(square.apply(5)); // 25
    }
}
```



Evaluación parcial de funciones

Al igual que Java, C# no brinda soporte nativo para la aplicación parcial de métodos. Sin embargo, es posible simular esta característica de manera muy natural utilizando delegados y lambdas.

```
using System;

public class Example
{
    public static void Main()
    {
        Func<int, int, int> power = (b, exp) => (int)Math.Pow(b, exp);

        Func<int, int> square = b => power(b, 2);

        Console.WriteLine(square(5)); // Imprime 25
    }
}
```



Evaluación perezosa

Java permite evaluación perezosa a través de la interfaz funcional Supplier y mediante la API de Streams.

```
public class EvaluacionEstricta {  
  
    static int calcular() {  
        System.out.println("Ejecutando cálculo...");  
        return 42;  
    }  
  
    static void metodoEstricto(boolean usarValor, int valor) {  
        if (usarValor) {  
            System.out.println("El valor es: " + valor);  
        } else {  
            System.out.println("No se utilizó el valor.");  
        }  
    }  
  
    public static void main(String[] args) {  
        metodoEstricto(false, calcular());  
    }  
}
```


Evaluación perezosa

Java permite evaluación perezosa a través de la interfaz funcional Supplier y mediante la API de Streams.

```
public class EvaluacionEstricta {  
  
    static int calcular() {  
        System.out.println("Ejecutar  
        return 42;  
    }  
  
    static void metodoEstricto(boolean  
        if (usarValor) {  
            System.out.println("El v  
        } else {  
            System.out.println("No s  
        }  
    }  
  
    public static void main(String[],  
        metodoEstricto(false, calcula  
    }  
}
```

```
import java.util.function.Supplier;  
public class EvaluacionEstricta {  
  
    static int calcular() {  
        System.out.println("Ejecutando cálculo...");  
        return 42;  
    }  
  
    static void metodoPerezoso(boolean usarValor, Supplier<Integer> pr) {  
        if (usarValor) {  
            System.out.println("El valor es: " + pr.get());  
        } else {  
            System.out.println("No se utilizó el valor.");  
        }  
    }  
  
    public static void main(String[] args) {  
        metodoPerezoso(false, () -> calcular());  
    }  
}
```



Evaluación perezosa

En C#, el equivalente exacto a un `Supplier<T>` de Java es un delegado `Func<T>` (un `Func` que solo tiene tipo de retorno y ningún parámetro de entrada)

```
using System;

public class EvaluacionPerezosaFunc{
    static int Calcular(){
        Console.WriteLine("Ejecutando cálculo...");
        return 42;
    }

    static void MetodoPerezoso(bool usarValor, Func<int> proveedorValor){
        if (usarValor){
            Console.WriteLine("El valor es: " + proveedorValor());
        }
        else{
            Console.WriteLine("No se utilizó el valor.");
        }
    }

    public static void Main(){
        MetodoPerezoso(false, () => CalculoCostoso());
    }
}
```



Evaluación perezosa

Java permite evaluación perezosa a través de Streams, útil para el procesamiento de colecciones.

```
import java.util.List;
import java.util.stream.Stream;

public class LazyStreamExample {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);

        Integer resultado = numbers.stream()
            .map(n -> {
                System.out.println("Mapping " + n);
                return n * 2;
            })
            .filter(n -> {
                System.out.println("Filtering " + n);
                return n > 5;
            })
            .findFirst()
            .orElse(-1);

        System.out.println("Resultado final: " + resultado);
    }
}
```

Mapping 1
Filtering 2
Mapping 2
Filtering 4
Mapping 3
Filtering 6
Resultado final: 6



Evaluación perezosa

Java permite evaluación perezosa a través de Streams, útil para el procesamiento de colecciones.

```
import java.util.stream.Stream;

public class InfiniteStreamExample {
    public static void main(String[] args) {
        Stream<Integer> infiniteStream = Stream.iterate(1, n -> n + 1);

        // Primeros 5 valores pares de forma perezosa
        infiniteStream
            .filter(n -> n % 2 == 0)
            .limit(5)
            .forEach(System.out::println);
    }
}
```

2
4
6
8
10



Evaluación perezosa

Y podemos procesar secuencias de elementos a través de `IEnumerable<T>` y LINQ.

```
using System;
using System.Collections.Generic;
using System.Linq;

public class StreamInfinitoEjemplo
{
    static IEnumerable<int> GenerarNumerosInfinitos()
    {
        int n = 1;
        while (true) {
            yield return n;
            n++;
        }
    }
    ...
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;

public class StreamInfinitoEjemplo
{
    ...
    public static void Main() {
        IEnumerable<int> streamPerezoso = GenerarNumerosInfinitos();

        var primeros5Pares = streamPerezoso
            .Where(n => n % 2 == 0)
            .Take(5);

        foreach (int numero in primeros5Pares) {
            Console.WriteLine(numero);
        }
    }
}
```



Python y JavaScript

Funciones de orden superior

Las **formas funcionales** pueden tomar una función como entrada y retornar una función como resultado. Nos permiten crear funciones más complejas a partir de las funciones primitivas.

Python provee algunas funciones de orden superior que resultan muy útiles para procesar secuencias de elementos. En particular, se destacan *map*, *sorted* y *filter*.

```
>> letras = ['a', 'b', 'c', 'd']

>> mayus = map(lambda x: x.upper(), letras)
>> mayus ⇒ <map object at ...>

>> mayus = list(map(lambda x: x.upper(), letras))
>> mayus ⇒ ['A', 'B', 'C', 'D']

>> vocales = list(filter(lambda x: es_vocal(x), letras))
>> vocales ⇒ ['a']
```



Funciones de orden superior

Las **formas funcionales** pueden tomar una función como entrada y retornar una función como resultado. Nos permiten crear funciones más complejas a partir de las funciones primitivas.

JavaScript provee algunas funciones de orden superior que resultan útiles para procesar secuencias de datos, como puede ser un arreglo.

```
const array = [1, 2, 3];
array.forEach(function(element) {
    console.log(elemento * 2);
});
```

```
const array = [1, 2, 3];
const mapeo = array.map(function(element) {
    return element + 1;
});
>> mapeo ⇒ [2, 3, 4]

const mapeo_bool = array.map(function(element) {
    return element > 1;
});
>> mapeo_bool ⇒ [false, true, true]

const filtrado = array.filter(function(element) {
    return element > 1;
});
>> filtrado ⇒ [2, 3]
```


Funciones como ciudadanos de primera clase

En Python las funciones se tratan como valores de primera clase, por lo que una función puede:

- Ser pasada como parámetro a otra función.
- Ser el resultado de otra función.
- Asignarla a una variable.
- Almacenarla en una estructura de datos.

No es exactamente igual al paradigma funcional
(efectos colaterales, transparencia referencial,
cuestiones de alcance)

```
def componer(f, g):  
    def h(x):  
        return f(g(x))  
    return h  
  
def cuadrado(x):  
    return x**2  
  
def por_tres(x):  
    return x*3  
  
>> myFunc = componer(cuadrado, por_tres)  
  
>> myFunc(2) ⇒ 36  
  
>> miLista = [1, 2, "hola", myFunc]  
>> miLista[3](2) ⇒ 36
```



Funciones como ciudadanos de primera clase

En JavaScript, las funciones son tratadas como cualquier otro valor: pueden pasarse como argumento, ser retornadas por otra función o ser asignadas a una variable.

```
function cuadrado(x) {  
    return x*x;  
}  
  
function doble(x) {  
    return x*2;  
}  
  
function calcular(fn, x) {  
    return fn(x);  
}  
  
function componer(f, g) {  
    function h(x) {  
        return f(g(x));  
    }  
    return h;  
}
```

```
const v1 = calcular(cuadrado, 5);  
>> v1 ⇒ 25  
  
const v2 = calcular(doble, 5);  
>> v2 ⇒ 10  
  
const v3 = doble(cuadrado(5));  
>> v3 ⇒ 50  
  
const v4 = cuadrado(doble(5));  
>> v4 ⇒ 100  
  
const v5 = componer(doble,  
cuadrado);  
>> v5(6) ⇒ 72
```

JS

Lambdas

Una lambda en Python es una función anónima, que se define usando la palabra *lambda*, seguida de los parámetros, dos puntos y una expresión.

Las lambdas están limitadas a una sola expresión y se usan comúnmente como argumentos de funciones de orden superior como map, filter o sorted.

```
square = lambda x: x * x
print(square(4))    # 16

suma = lambda x, y: x + y
print(suma(3, 5))   # 8
```

```
nums = [1, 2, 3, 4]
cuadrados = list(map(lambda x: x**2, nums))
print(cuadrados)    # [1, 4, 9, 16]

nums = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, nums))
print(pares)        # [2, 4, 6]

palabras = ["hola", "adiós", "bien", "zorro"]
ordenadas = sorted(palabras, key=lambda x: len(x))
print(ordenadas)    # ['bien', 'hola', 'zorro', 'adiós']
```



Lambdas

JavaScript utiliza **funciones flecha** ($\Rightarrow$) como su versión de las lambdas. Ofrecen una sintaxis más corta para definir funciones. Se pueden usar en cualquier lugar donde se usen funciones normales.

Las funciones flecha no tienen su propio *this*, lo cual puede resultar más natural de entender.

```
const square = x => x * x;

console.log(square(4)); // 16

const add = (a, b) => {
  return a + b;
};

console.log(add(2, 3)); // 5
```

```
function Normal() {
  this.name = "Normal";
  setTimeout(function() {
    console.log(this.name); // undefined
  }, 1000);
}

function Arrow() {
  this.name = "Arrow";
  setTimeout(() => {
    console.log(this.name); // "Arrow"
  }, 1000);
}
```

Evaluación parcial de funciones

A pesar de no contar con currying, Python permite utilizar un mecanismo de aplicación parcial a partir de la función *partial* de la librería *functools*.

```
from functools import partial

def multiplicar(x, y):
    return x * y

# crear función para multiplicar por dos
>> doble = partial(multiplicar, y = 2)

>> doble(5) ⇒ 10

# crear función para multiplicar por tres
>> triple = partial(multiplicar, x = 3)

>> triple(y = 10) ⇒ 30
```



Evaluación parcial de funciones

Otra manera de hacer aplicación parcial es definiendo *closures*, que es una función que retiene el ambiente del entorno que la contiene (usualmente otra función), aún cuando ese entorno ha terminado su ejecución.

```
def counter():  
    count = 0  
  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
  
    return increment  
  
c = counter()  
print(c()) # 1  
print(c()) # 2  
print(c()) # 3
```

```
def h(x):  
    return lambda y: x + y  
  
a = h(1)  
b = h(10)  
  
print(a(1)) # 2  
print(b(1)) # 11
```



Evaluación parcial de funciones

En JavaScript también existen librerías que nos ayudan a definir funciones sobre las que pueden hacerse aplicaciones parciales, y pueden definirse *closures*.

```
import * as R from 'ramda'

const data1 = ['A', 'B', 'C'];
const data2 = ['E', 'F', 'G'];

>> R.nth(1, data1) ⇒ 'B'

const getValorPosicion = R.nth(1)

>> getValorPosicion(data1) ⇒ 'B'
>> getValorPosicion(data2) ⇒ 'F'
```

```
function counter() {
  let count = 0;
  return function increment() {
    count++;
    return count;
  };
}

const c = counter();
console.log(c()); // 1
console.log(c()); // 2
console.log(c()); // 3
```

JS

Evaluación perezosa

Una de las facilidades importantes que utiliza evaluación perezosa está relacionado con las funciones y expresiones generadoras.

En Python, estas expresiones generan un iterador que nos permite recorrer los elementos de una estructura (por ejemplo, una lista potencialmente infinita) bajo demanda, dando un elemento a la vez.

```
def Naturals():  
    x = 0  
    while True:  
        yield x  
        x += 1  
  
>> s = (x for x in Naturals()  
         if x**2 > 20 and x % 2 == 0)  
  
>> next(s) ⇒ 6  
  
>> next(s) ⇒ 8
```



A diferencia de *return*, *yield* permite mantener en memoria el estado de las variables locales de la función y el punto de ejecución en cada momento.

Evaluación perezosa

En JavaScript también podemos hacer uso de funciones generadoras.

Estas funciones devuelven un objeto de tipo Generator, que es una clase de iterador al cual le podemos pedir cada elemento de la secuencia bajo demanda.

```
function* naturales() {  
  var index = 0;  
  while(true)  
    yield index++;  
}  
  
>> const gen = naturales();  
  
>> gen.next()           ⇒ { value: 0, done: false }  
>> gen.next().value ⇒ 0  
>> gen.next().value ⇒ 1  
>> gen.next().value ⇒ 2
```

JS

¿Preguntas?
